

信息科技部 · AI 赋能创新实践汇报

一个人 + AI = 一个生产力团队

DBA 专家驱动 AI 协作，从零构建企业级 SQL 审核平台

汇报人：林桑（DBA） 项目：TDSQL 数据库 SQL 审核工具 v1.0.2

8 周

从零到上线

119 条

审核规则

880 项

自动化测试

0 行

本人手写代码

汇报目录

01

背景与挑战

最懂需求的人，不会写代码

02

方法论

专家驱动的三角色 AI 协作模型

03

实战历程

8 周、62 次提交、从零到 v1.0.2

04

提示词工程

让 AI 完全理解人的意图

05

成果与数据

效率提升 3-6 倍，节省 50 万 +

06

能力提升与推广

AI 是放大器，专家是灵魂

01

背景与挑战

SQL 质量管控之痛，与一个 DBA 的破局思考

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

日常运维中的三大核心痛点

SQL 质量管控缺失

开发提交的 SQL 未经审核直接上线，生产事故频发

影响：系统稳定性差 · 运维成本高

Oracle 迁移适配难

业务系统 Oracle → TDSQL，42 类不兼容语法需逐一排查

影响：人工审查效率极低 · 遗漏风险大

慢 SQL 治理被动

缺乏主动发现与优化建议能力，问题暴露后才能救火

影响：用户体验差 · 资源浪费

痛点的共同根源：**SQL 质量问题必须在上线前拦截，而拦截需要一套自动化、懂 TDSQL 的审核工具。**

三条传统路径，条条走不通



采购商业产品

年费数十万，定制化困难，与行内 TDSQL 环境适配差



外包开发

沟通成本高、交付周期长，后续维护受制于人



自行开发

我是 DBA，数据库运维专家——但编程一窍不通

核心矛盾

最懂业务需求的人（DBA）不具备编程能力；有编程能力的人，不理解 DBA 的专业需求。

破局思路：一个大胆的假设



我懂 **TDSQL** 的每一个技术细节和运维规范



我能设计出完整的审核规则体系和架构方案



AI 工具可以把我的设计思想转化为可运行的代码

大胆假设：一个人 + **AI** 工具 = 一个完整的生产力团队

02

方法论：专家驱动 AI 协作模型

把 AI 当团队管理，而不是当工具使用

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

核心理念：专家是「大脑」，AI 是「双手」

专家 = 大脑

- 架构设计能力 —— 定方向、定边界、定标准
- 业务理解深度 —— 懂 TDSQL 的每一个细节
- 质量判断标准 —— 什么是「生产级可用」

提供方向和质量标准

AI = 双手

- 代码实现能力 —— 万行级工程的落地执行
- 快速迭代能力 —— 小时级完成一轮整改
- 多角色协同能力 —— 设计 / 编码 / 测试并行

提供执行力和生产效率

关键洞察：我不是在和 AI 「对话」，我是在管理一个 AI 团队 —— 像 Tech Lead 管理开发团队一样。

三角色 AI 协作架构：各司其职、相互制衡



为什么要三个角色？ 单一 AI 既当运动员又当裁判必然失控。设计出标准、编码去实现、测试来揭短，我做最终裁决 —— 制衡产生质量。

五步交互方法论：数十轮迭代沉淀的固定打法



🔄 循环迭代，直到质检通过准出 —— 本项目最多的一个模块循环了 **4** 轮，最终 **0** 缺陷上线

这套流程看似「慢」，实际是最快的路径：设计先行避免了方向性返工，独立测试把缺陷拦在上线前。

质量的秘密：制衡，而不是信任



真实一幕：编码智能体报告「已完成并自测通过」，质检验收却发现 11 个页面是空壳——
「AI 说做完了」不算数，独立验证 + 专家裁决才算数。

03

实战历程：从零到上线

8 周 · 7 个阶段 · 62 次提交 · 一次真实的完整交付

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

项目全景：8 周走完传统 6-12 个月的路



每个阶段，我扮演不同的专家角色；AI 团队则始终保持三角色并行 —— 这是一个人的「多线程」工作方式。

工程全景数据（源自 Git 仓库真实统计）

62

次代码提交

全部由 AI 编写并提交

2.98 万

行工程代码

后端 1.65 万 + 测试 1.18 万 +
前端

26 份

项目文档 1.25 万行

设计 / 测试 / 部署 / 运维全套

880 项

自动化测试用例

上线前全量回归 0 失败

119 条审核规则 · 9 大分类

含 42 条 Oracle 迁移兼容规则，覆盖原厂规范 52 个规范点 100% 无遗漏

14 个后端模块 · 19 个前端页面

认证 RBAC / 多实例注册 / 规则集 / 门禁 / 慢 SQL / 体检 / 监控 / 巡检...

我本人手写代码：0 行

我贡献的是：设计思想、规则体系、每一轮的质量裁决与验收标准

Oracle 迁移规则的诞生：一份 729 行的原厂文档

背景

原厂交付《Oracle 迁移 TDSQL 改造适配方案》V1.5.1（729 行 .doc），涉及：

- to_char / nvl / decode 等函数不支持，需替换改写
- rownum / MERGE INTO / WITH AS 等语法不兼容
- 分布式场景：窗口函数、游标、临时表限制
- 分片键六条军规（类型 / 唯一索引 / 更新限制…）

如果靠人工

逐条记忆 + 人工排查

一个 **DBA** 团队需要数月

且必然存在遗漏风险

我的目标只有一个：把原厂文档的每一个规范点，无遗漏地变成审核工具里可执行的规则。

六步走完：从原厂文档到 42 条可执行规则

1 我下达指令

把 729 行文档交给设计智能体：「每一个规范点无遗漏映射到规则体系，标注映射 / 新增」

2 AI 出设计

设计智能体产出 648 行设计说明书：52 点追溯矩阵 + 42 条规则逐条规格 + 测试设计

3 我再委派

把设计说明书交给编码智能体：「严格按每一个规格编码，不得遗漏任何一条」

4 AI 编码

oracle_compat.py 978 行、42 个规则类、解析器增强、前端分类展示

5 独立揭短

测试 + 质检发现 3 个真缺陷：UNSIGNED 误报、R054 死代码、断言未更新

6 我裁决闭环

逐条确认根因 → 指挥修复 → 184 项复验全过 → 880 回归 0 失败，准出

关键洞察

如果我不理解 **TDSQL** 的技术细节和原厂文档的每一个规范点，**AI** 不可能自动完成这些工作。
是我的专家判断力，确保了规则的完整性与准确性。

成果：数月的工作量，一周交付

42 条

新增审核规则

R078 - R119

52 / 52

原厂规范点覆盖

100% 无遗漏追溯

184 项

规则级验收检查

全部通过

≈1 周

全流程耗时

传统方式需数月

上线后的每一次 SQL 审核，这 42 条规则都在自动工作：

开发再提交 **to_char / nvl / rownum / merge into** 等 Oracle 写法，审核工具当场拦截，并给出原厂文档对应的 TDSQL 改写方案。

—— 原厂规范从「纸面文档」变成了「每一次提交都在执行的自动化防线」。

质检验收闭环：AI 给 AI 挑刺，我做裁决

质检智能体的验收结论（原文）

「引擎本体是几轮交付里质量最高的……但仍不能准出：1 个军规是死代码、1 个高频误报、测试工作完全没做。」

我的处理（专家裁决）

- 逐条审核 4 个缺陷，确认根因分析准确
- 按优先级下达整改指令给编码智能体
- 要求复验口径：4 条自测 SQL + 全量回归 0 失败

✓ R117 误报

5 行代码修复

✓ R054 死代码

解析器 + 规则双层修复

✓ 测试未做

103 条新单测补齐

✓ 文档口径

6 处更新

复验结果：880 passed · 0 failed，准出上线。

制衡的核心：编码智能体说「写对了」不算数——必须过独立验证，再由我做最终裁决。

「无数次交互」的真实样貌：前端 V3.0 的四轮攻坚

第 1 轮

验收发现 19 个页面中 11 个是「空壳」——提交说明与实际严重不符

我发整改单：逐页列出问题 + 后端接口清单

第 2 轮

页面填上了，但表格全空。质检定位根因：响应数据「信封」未拆包

我要求「傻瓜式」整改文档：逐行给出错误代码 → 正确代码

第 3 轮

12 处解包全部修复，仅剩 2 个写接口参数格式错误

我给出照抄级修复代码，并要求「亲手点一遍再交付」

第 4 轮

全部修复验证通过，RBAC 顺手校正 —— 验收通过

四轮不是失败，是方法论在工作：每一轮都可度量、可收敛

整个项目中，这样的「验收 → 整改 → 复验」循环发生了十余次 —— 交互越充分，AI 的输出越接近专家标准。

04

提示词工程

让 AI 完全理解人的意图 —— 六条铁律与四个实录

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

与 AI 高效协作的六条铁律

铁律 1 专家先行

先想清楚再让 AI 做，不让 AI 替你做决策

反例：「帮我设计一个系统」→ 太模糊

铁律 2 设计先行

先出设计说明书，再让编码智能体实现

反例：「直接写代码」→ 方向偏差

铁律 3 精确约束

给出明确的验收标准与边界条件

反例：「优化一下性能」→ 无标准

铁律 4 闭环验证

每一轮交付必须经过独立测试验证

反例：「写完就提交」→ 质量失控

铁律 5 迭代改进

接受 AI 不完美，持续推动收敛

反例：「一次就要完美」→ 不现实

铁律 6 角色分工

设计 / 编码 / 测试各司其职，不混岗

反例：「让编码的做测试」→ 无制衡

一句话总结：把 AI 当成能力极强但需要明确指令的新员工 —— 你给的目标越清晰、验收越严格，它的产出越接近专家水准。

提示词实录（一）：任务委派 与 缺陷修复

模式一 · 任务委派

“你从 GitHub 更新一下最新内容，然后把设计说明书的内容严格按照要求进行编码开发。设计说明书位置在 docs/xxx.md，你自己取一下。

完成后自行开展一轮冒烟测试，冒烟测试通过后向我汇报，我会让测试智能体进行 SIT 和 UAT 测试。”

要点：明确输入（设计说明书）、明确流程（编码→冒烟→汇报）、明确后续（测试智能体接手）。

模式二 · 缺陷修复

“这是测试智能体的测试报告，你照着修复。
改好后我让测试智能体复测。”

（把测试报告原样传递，不做二次转译）

要点：直接传递测试报告让 AI 自行分析修复，不过度指导；「复测」预告形成压力闭环。

提示词实录（二）：质检驱动与需求定义

模式三 · 质检驱动整改

“我让设计智能体开展了一轮质检验收，结论是
不予准出，质检意见在 docs/xxx.md，你拿下来
照着改一下。”

（质检报告里每个缺陷带：根因定位 + 精确到
行号的修复指引 + 修完的自测 SQL）

要点：引入独立质检角色，用质检报告驱动修复；整改单越「傻瓜式」，修复越精准。

模式四 · 需求定义

“原厂今天给过来一份文档《Oracle 迁移 TDSQL 改造
适配方案》，给出了一些 TDSQL 使用规范，比如
to_char 函数不支持需要替换改写等。这些内容对
使用 TDSQL 的业务系统至关重要，现在急需把文档
内容全部无遗漏地添加到审核规则库中。”

要点：说清业务背景、紧迫程度、核心要求（无遗漏）——让 AI 理解「什么最重要」。

踩过的坑：五大陷阱与正确做法

陷阱	后果	正确做法
⚠ 让 AI 自己做架构决策	架构混乱、前后矛盾	✓ 专家定义架构，AI 实现细节
⚠ 一次提太多需求	理解偏差、遗漏需求	✓ 分阶段迭代，每阶段聚焦
⚠ 不看 AI 输出就继续	错误累积、返工成本高	✓ 每轮审核后再下达下一轮指令
⚠ 忽略独立测试	上线后才发现缺陷	✓ 三角色制衡，测试先行
⚠ 硬用编程术语沟通	词不达意、来回返工	✓ 用业务语言描述需求即可

05

成果与数据

一款可上线的企业级产品，和一笔算得清的账

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

交付成果：TDSQL 数据库 SQL 审核工具 v1.0.2

SQL 即时审核

文件批量审核

GitLab 集成审核

慢 SQL 扫描分析

EXPLAIN 诊断

数据库体检

大表治理

质量门禁

规则集多租户

角色权限 RBAC

监控告警

操作审计

数据保留策略

Prometheus 指标

一键部署回滚

规则

119 条审核规则 · 9 大分类，含 42 条 Oracle 迁移兼容规则（覆盖原厂 52 点 100%）

质量

880 项全量回归 0 失败 · 184 项规则验收全通过 · 多轮 SIT/UAT + 独立质检准出

交付

麒麟 V10 SP3 一键部署包（离线安装 / 预检 / 冒烟 / 回滚） + 13 份 v1.0.2 文档体系

效率对比：同样的活，时间轴完全不同



总周期：传统方式 6-12 个月 → AI 协作 8 周 (整体提速 3-6 倍)

成本对比：一笔算得清的账

采购商业产品

30-80 万 / 年

3-6 月选型适配

外包开发

50-150 万

6-12 月交付

自建团队（3-5 人）

100-200 万 / 年

6-12 月

我 + AI 工具

0 额外人力成本

8 周交付

直接效益（最保守口径）：节省 **≥ 50 万元** 开发成本 + **6 个月** 时间成本

长期效益：SQL 缺陷在上线前拦截，生产事故与救火成本持续下降（难以估量但真实存在）

06

能力提升与推广建议

AI 放大了专家能力，而不是替代了专家能力

背景与挑战

方法论

实战历程

提示词工程

成果与数据

能力提升与推广

AI 如何提升了我：四个维度的能力跃迁

从 **DBA** 到全栈交付者

只会 SQL 运维



能独立交付完整软件产品

从执行者到架构师

执行运维任务



设计系统架构、指挥 **AI** 团队

从个人到团队

一个人的能力边界



一个人 = 一个生产力团队

从被动到主动

问题来了再救火



主动构建工具预防问题

为什么必须「专家驱动」：五项不可替代的能力

1 需求定义能力

「我知道 DBA 真正需要什么工具」

2 架构设计能力

「我能设计出 119 条规则的完整体系」

3 质量判断能力

「我能判断 AI 的输出是否达到生产级」

4 业务理解深度

「我理解 Oracle→TDSQL 迁移的每一个技术细节」

5 优先级决策能力

「我知道什么最重要，什么可以延后」

这五项能力，AI 一项都替代不了 —— 它们恰恰是每一位领域专家已经具备的。

方法论完全可复制：从一个人的实践到一个部门的能力

任何领域专家都可复用

DBA：SQL 审核 / 慢 SQL 平台 / 巡检系统

网络工程师：网络监控 / 故障自动诊断

安全工程师：漏洞扫描 / 合规检查平台

运维工程师：CMDB / 自动化运维 / 告警系统

业务分析师：数据分析工具 / 报表平台

对部门的五点推广建议

机制：建立「AI+ 专家」项目孵化机制

培训：开展提示词工程培训，推广五步方法论

工具：统一 AI 平台，建立智能体协作规范

激励：AI 协作创新纳入绩效，设创新实践奖

知识：建案例库，沉淀方法论与提示词模板

结语

这不是一个程序员的故事，
而是一个 **DBA** 用专家智慧指挥 **AI** 团队的故事。

我没有写一行代码，但我设计了 119 条规则的完整体系，定义了 42 条迁移规范的技术细节，
审核了每一轮测试报告，并做出了每一次质量裁决。

AI 是放大器 —— 它放大了专家能力，而不是替代了专家能力。

一个人 + **AI** = 一个生产力团队。这不是未来的愿景，而是今天正在发生的现实。

—— 感谢聆听 ——